

- Realistisk modellering leder ofte til regnekrevende operasjoner der problemene (ligningene) stort sett ikke har eksakt (analytisk) løsning, og må derfor løses numerisk (tilnærmet)
- Og selv om analytiske løsninger foreligger, så er de ofte så kompliserte at de er ubrukelig hvis vi bare er ute etter tilnærmelser¹
- Vi skal derfor gi og bruke metoder for å løse mange forskjellige viktige problemstillinger som ofte opptrer i anvendelser
- Vi skal oppøve forståelse for metoders begrensninger og muligheter

¹Men, det må understrekes at analytiske teknikker kan være svært viktige for å analysere et problem for bla. å skreddersy en løsningsmetode, som kan være numerisk.

Hva kjennetegner en god numerisk metode?

- Tilpasset nøyaktighet (mulighet til å estimere, kontrollere og monitorere nøyaktighet (adaptivitet))
- Robusthet/stabilitet
- Regneeffektivitet

||
||



Økningen i regneeffektivitet har vært og vil bli sterkt avhengig av både bedre algoritmer og raskere maskiner.

LÆRINGSMÅL

- Evaluering av polynom (bla. Horners metode) (S: 0.1)
- Binære tall - konvertering mellom desimaltall og binærtall (begge veier) (S: 0.2)
- Andre tallsystem (S: 0.2)
- Flyttall: Representasjon, formater og beregninger (S: 0.3)

monomer : x^k $k \geq 0$
 $c \cdot x^k$

$k > 0$
 $c \cdot x^1$
 $c \cdot x^2$
 $\vdots$
 $c \cdot x^{100}$

1. Evaluering av polynom (bla. Horners metode)
2. Binære tall - konvertering mellom desimal tall og binære tall (begge veier)
3. Andre tallsystem
4. Reelle tall representert som flyttall (del 1)

$k = 0$
 $x^0 = 1$
 $c \cdot 1$

A $e^{4x} + x + 7$

C 4 grad 0

B $4x$ + x^2 + 7
 grad 2

D t^1 - 1
 grad 1

$$\sin x + x \quad x^5 + x^3 + 1$$

Finne polynomverdier Naive metoder og Horner

Eksempel $P(x) = 3x^4 - 2x^3 - x^2 - 3x + 3$. Regn ut $P(x)$ for $x = 2$

Naiv metode Skriver ut alle potenser og regner ut

$$P(2) = 3 \cdot 2 \cdot 2 \cdot 2 \cdot 2 - 2 \cdot 2 \cdot 2 \cdot 2 - 1 \cdot 2 \cdot 2 - 3 \cdot 2 + 3 = 25$$

(10 multiplikasjoner 4 addisjoner.)

Generelt: $\frac{d^2+3d}{2}$ regneoperasjoner.)

Litt mindre naiv metode Gjenbrukes potenser for å regne ut neste

$$\text{potens } 2^2 = 2 \cdot 2 = 4, 2^3 = 2^2 \cdot 2 = 8, 2^4 = 2^3 \cdot 2 = 16$$

$$P(2) = 3 \cdot 16 - 2 \cdot 8 - 1 \cdot 4 - 3 \cdot 2 + 3 = 25$$

(7 multiplikasjoner 4 addisjoner.)

Generelt: $3d - 1$ regneoperasjoner.)

Horners metode Kommer på neste side (4 multiplikasjoner 4 addisjoner.)

Generelt: $2d$ regneoperasjoner.)

$P(2)$

$$d^2 + 3d$$

2

$$3d - 1$$

$$2d$$

Her er d polynomets grad.

Horner's metode

Minimerer antall utregner ved å nøste opp polynomet.

$$\begin{aligned}
 P(x) &= a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 \\
 &= (a_4x^3 + a_3x^2 + a_2x + a_1)x + a_0 \\
 &= ((a_4x^2 + a_3x + a_2)x + a_1)x + a_0 \\
 &= (((a_4x + a_3)x + a_2)x + a_1)x + a_0
 \end{aligned}$$

$$P(2) = (((3 \cdot 2 - 2) \cdot 2 - 1) \cdot 2 - 3) \cdot 2 + 3 = 25$$

(4 multiplikasjoner 4 addisjoner)

2 d

Polynomer med basepunkter er viktig i senere kapitler

$$Q(x) = (((a_4(x - b_4) + a_3)(x - b_3) + a_2)(x - b_2) + a_1)(x - b_1) + a_0$$

Kommer tilbake til bruken av basepunkter når det er aktuelt.

Hvorfor er det å minimalisere antall regneoperasjoner så viktig?

EKSEMPEL (gjentatt med flere detaljer enn det som var tilfelle ovenfor)

Gitt $p(x) = 3x^4 - 2x^3 - x^2 - 3x + 3$, bestem $p(2)$!

$$p(x) = (((3x - 2)x - 1)x - 3)x + 3$$

$$[3, -2, -1, -3, 3]$$

$$3 \times 2 - 2 = 4$$

$$4 \times 2 - 1 = 7$$

$$7 \times 2 - 3 = 11$$

$$11 \times 2 + 3 = 25$$

Så, $p(2) = 25$ i overensstemmelse med tidligere beregning.

Som vi ser, jobber vi fra innerste nivå og utover.

```

1 #Evaluate value of polynomial using Horner's method
2 def horner(poly, x):
3     n = len(poly)
4     result = poly[0]
5     for i in range(1, n):
6         result = result * x + poly[i]
7     return result

```

$$\underline{2x^2 + 3x + 1 = (2x+3) \cdot x + 1}$$

$$4x^3 + 3x^2 + 7x + 5 =$$

$$= (4x^2 + 3x + 7) \cdot x + \underline{5} =$$

$$= \left((4x + 3) \cdot x + 7 \right) \cdot x + 5$$

* + * + * +

antall ops
 $(d-1) \cdot 2 + \underbrace{2}_{\text{utenv. par.}} = 2 \cdot d$
 antall parent.

1. Evaluering av polynom (bla. Horners metode)
2. Binære tall - konvertering mellom desimal tall og binære tall (begge veier)
3. Andre tallsystem
4. Reelle tall representert som flyttall (del 1)

$$(1034.82)_{10} = \underbrace{1}_{10^3} \cdot 10^3 + \underbrace{0}_{10^2} \cdot 10^2 + \underbrace{3}_{10^1} \cdot 10^1 + \underbrace{4}_{10^0} \cdot 10^0 + \underbrace{8}_{10^{-1}} \cdot 10^{-1} + \underbrace{2}_{10^{-2}} \cdot 10^{-2}$$

Først et tilbakeblikk på 10-tallsystemet som vi er vant til for å se på hvordan vi bestemmer hvert siffer i et heltall der:

$$1034 = 103 \times 10 + \textcircled{4}$$

$$103 = 10 \times 10 + \textcircled{3}$$

$$10 = 1 \times 10 + \textcircled{0}$$

$$1 = 0 \times 10 + \textcircled{1}$$

Vi får altså her sifrene i 1034 i **motsatt** rekkefølge.¹

$$(1034)_{10} = 1 \cdot \underline{10^3} + 0 \cdot \underline{10^2} + 3 \cdot \underline{10^1} + 4 \cdot \underline{10^0}$$

$$(1034)_8 = 1 \cdot 8^3 + 0 \cdot 8^2 + 3 \cdot 8^1 + 4$$

¹Heltallsdivisjonen $1034/10$ gir kvotienten 103 og resten 4 osv..

$$(b_2 b_1 b_0 . b_{-1} b_{-2})_2$$

Binær-representasjoner er

$$\dots b_2 2^2 + b_1 2^1 + b_0 2^0 + b_{-1} 2^{-1} + b_{-2} 2^{-2} \dots$$

for $b_j = 0$ eller 1 for alle $j = \dots, -1, 0, 1, \dots$

IMAx2150: Prinsippet bak bestemmelsen av binær-representasjonen til et **heltall**

Gitt det positive heltallet n skrevet i 10-tallssystemet der vi skal bestemme dets binær-representasjon som $n = (\underline{b_n b_{n-1} \cdots b_3 b_2 b_1})_2$ der $b_j = 0$ eller 1 for alle $j \geq 0$.

Vi har at $n = 2t + b_1$ for t heltall. (Hvorfor?)

Da gir $n/2$ kvotient t og rest b_1 (b_1 er jo 0 eller 1).

Fortsetter vi på samme måte med t i stedet for n , så gir $t/2$ rest b_2 , osv..

Dette gjentas til alle bitene i binær-representasjonen av n er bestemt.¹

EKSEMPEL:

$$n = 7 = (\mathbf{111})_2:$$

$$7/2 = \mathbf{3} \times 2 + \mathbf{1}: \text{Heltallsdel mod 2 (kvotienten): } (\mathbf{11})_2 \text{ pluss rest mod 2: } \mathbf{1}$$

$$3/2 = \mathbf{1} \times 2 + \mathbf{1}: \text{Heltallsdel mod 2: } (\mathbf{1})_2 \text{ pluss rest mod 2: } \mathbf{1}$$

$$1/2 = 0 \times 2 + 1: \text{Heltallsdel mod 2: } (\mathbf{0})_2 \text{ pluss rest mod 2: } \mathbf{1}$$

$$0/2 = 0 \times 2 + 0 = (0)_2 + 0, \text{ og vi stopper}$$

(da flere 0-er her ikke endrer heltallet).

¹Merk rekkefølgen! Den **minst signifikante biten** bestemmes først, så den **nest minste signifikante biten** osv..

$$(9)_{10} \rightarrow (\underline{1\ 0\ 0\ 1})_2$$

$$9 = 4 \cdot 2 + \underline{1}$$

$$4 = 2 \cdot 2 + 0$$

$$2 = 1 \cdot 2 + 0$$

$$1 = 0 \cdot 2 + \underline{1}$$

IMAx2150: Prinsippet bak bestemmelsen av binær-representasjonen av en ekte brøk

$$f = (.b_1 b_2 \dots b_n)_2$$

I forhold til det vi gjorde når vi konverterte heltall, innser vi at for en **ekte brøk** f må vi **bytte ut** $/2$ med $\times 2$ for å få binær-representasjonen av brøken.

$$\text{La } f = \underbrace{b_1/2^1 + b_2/2^2 + b_3/2^3 \dots b_n/2^n + \dots}_{= 2f} = \underbrace{b_1 + (b_2/2^1 + b_3/2^2 \dots)}_{\text{brøk}}.$$

Høyre side består derfor av en heltallsdel b_1 og en ny rasjonal del (brøk).

Gjentas prosessen med å multiplisere denne rasjonale delen med 2, så får vi den neste heltallsdelen, dvs. b_2 .

Slik plukker vi bitene i f én etter én.

Proessen gjentas til den brøkdelen blir 0 (**eller begynner å gjenta seg**) (neste eksempel).

EKSEMPEL:

$$f = 1/8 = \underline{0.001}_2:$$

$$2f = .(01)_2 + 0 = g + \underline{0} = b_1$$

$$2g = .(1)_2 + 0 = h + 0 = b_2$$

$$2h = .(0)_2 + \underline{1}, \text{ og vi stopper med bitene } 001.^1$$

$$\begin{aligned} f &= 1/8 & g &= 2f = 1/4 = \underline{x.yz}f \\ h &= 2g = 2 \cdot 1/4 = 1/2 \\ 2 \cdot h &= 2 \cdot 1/2 = 1 \end{aligned}$$

¹Her får vi bitene i riktig rekkefølge i motsetning til når vi konverterer heltall.

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^{-1} = \frac{1}{2}$$

$$2^{-2} = \frac{1}{4}$$

$$2^{-3} = \frac{1}{8}$$

$$2^{-4} = \frac{1}{16}$$

$$\left(\frac{1}{16} + \frac{1}{32} \right)_{10}$$

$$\text{"}$$

$$(0.00011)_2$$

$$\frac{1}{16} = 1 \cdot \frac{2^{-4}}{1}$$

$$\left(\frac{1}{16} \right)_{10} = \left(\underline{0.000 \overset{\downarrow}{1}} \right)_2$$

$$x = 0.7$$

Fractional part. Convert $(0.7)_{10}$ to binary by reversing the preceding steps. Multiply by 2 successively and record the integer parts, moving away from the decimal point to the right.

$$\begin{array}{rcl}
 .7 \times 2 = .4 + 1 & = & 1 + 0.4 \\
 .4 \times 2 = .8 + 0 & = & 0 + 0.8 \\
 .8 \times 2 = .6 + 1 & = & 1 + 0.6 \\
 .6 \times 2 = .2 + 1 & = & 1 + 0.2 \\
 .2 \times 2 = .4 + 0 & = & 0 + 0.4 \\
 .4 \times 2 = .8 + 0 & = & 0 + 0.8 \\
 & \vdots &
 \end{array}$$

Notice that the process repeats after four steps and will repeat indefinitely exactly the same way. Therefore,

$$(0.7)_{10} = (.1011001100110\dots)_2 = (.1\overline{0110})_2,$$

Merk at for $0.7 = 7/10 = 7/(2 \times \textcircled{5})$ er **nevneren ikke en toer-potens** i motsetning til i vårt første eksempel $f = 1/8$ der binær-representasjonen derfor er endelig.

Dvs. vi har en **uendelig gjentakende periode**, som vi vet fra en setning vi skal se på senere.

La oss kontrollere svaret!

La $x = (0.\overline{10110})_2$.

Da er $2x = 1.\overline{0110}$.

Med $y = \overline{0110}$ er $2^4 y = \underbrace{(0110)}_2 + y$, så $y = 6/15 = 2/5$.

M.a.o. er $2x = 1 + y = 1 + 2/5 = 7/5$, og $x = 7/10 = 0.7$, **som skulle vises**.

Det er alltid lurt å **kontrollere** slike beregninger!

$$y = 0.0110 \underbrace{0110}_{\times 2^4} 0110 \dots$$

$$16y = 6 + y$$

$$x = (0.7\overline{34})_{10}$$

$$10x = 7.\overline{34}$$

$$y = 0.\overline{34}$$

$$100y = 34 + y$$

$$99y = 34$$

$$y = \frac{34}{99}$$

$$y = 0.3\overline{43434}$$

$$100y = \underline{\underline{34.343434..}}$$

Merk at for $0.7 = 7/10 = 7/(2 \times \textcircled{5})$ er **nevneren ikke en toer-potens** i motsetning til i vårt første eksempel $f = 1/8$ der binær-representasjonen derfor er endelig.

Dvs. vi har en **uendelig gjentakende periode**, som vi vet fra en setning vi skal se på senere.

La oss kontrollere svaret!

La $x = (0.\overline{10110})_2$.

Da er $2x = 1.\overline{0110}$.

Med $y = \overline{0110}$ er $2^4 y = \underbrace{(0110)}_2 + y$, så $y = 6/15 = 2/5$.

M.a.o. er $2x = 1 + y = 1 + 2/5 = 7/5$, og $x = 7/10 = 0.7$, **som skulle vises**.

Det er alltid lurt å **kontrollere** slike beregninger!

$$y = 0.0110 \underbrace{0110}_{\times 2^4} 0110 \dots$$

$$16y = 6 + y$$

Merk at i tabellen nedenfor må **siste kolonne leses baklengs**.

Antall (signifikante) **biter** i binær-representasjonen av 1019 er $\lfloor \log_2(1019) \rfloor + 1 = 10$.¹

n	n/2	Integer part module 2	+	Rest module 2
1019	1019/2	509	+	1
509	509/2	254	+	1
254	254/2	127	+	0
127	127/2	63	+	1
63	63/2	31	+	1
31	31/2	15	+	1
15	15/2	7	+	1
7	7/2	3	+	1
3	3/2	1	+	1
1	1/2	0	+	1

Binær-representasjon av 1019: $(\mathbf{1111111011})_2$

¹I python kan også `len(bin(1019)[2 :])` brukes.

Merk at her er det en **preperiode** på én bit før (den gjentakende) **perioden** på fire biter.¹

	n	Rasjonal del av 2^n	Heltallsdel av 2^n
2	0.1	0.2	0
2	0.2	0.4	0
2	0.4	0.8	0
2	0.8	0.6	1
2	0.6	0.2	1
2	0.2	0.4	0

) preperiode

Binær-representasjon av 0.1: $(0.00011)_2$

¹Preperioden kommer av faktoren 2^1 av $10 = 2^1 \times 5$ av nevneren til $1/10 = 0.1$. (Her kunne vi også ha forutsett at antall biter i perioden må være 4, men hvordan, det går vi ikke inn på).

1. Evaluering av polynom (bla. Horners metode)
2. Binære tall - konvertering mellom desimal tall og binære tall (begge veier)
3. Andre tallsystem
4. Reelle tall representert som flyttall (del 1)

Eksempel 141.4:

$$(141.4)_{10} = (\textcolor{red}{1000}\textcolor{red}{1101}.\overline{0110})_2 = (8D.\bar{6})_{16} = (215.\overline{3146})_8.^1$$

Det **heksadesimale tallsystemet** (16-talls-systemet):

Hex	Dec	Bin	Hex	Dec	Bin
0	0	0000	<u>8</u>	8	<u>1000</u>
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

$$(1111)_2 = (15)_{10} = (F)_{16}$$

¹Bestem selv $(141)_2$ og $(0.4)_2$. $(141.4)_8$ følger av å gruppere bitene på tre og tre, dvs. $(141.4)_8 = |010|001|101|.\overline{|011|001|100|110|}$.

Eksempel 141.4:

$$(141.4)_{10} = (10001101.0110)_2 = (8D.\bar{6})_{16} = (215.\overline{3146})_8.^1$$

Det **heksadesimale tallsystemet** (16-talls-systemet):

Hex	Dec	Bin	Hex	Dec	Bin
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

Det **oktadesimale tallsystemet** (åtte-talls-systemet):

Hex	Dec	Bin	Hex	Dec	Bin
0	0	000	4	4	100
1	1	001	5	5	101
2	2	010	6	6	110
3	3	011	7	7	111

¹Bestem selv $(141)_2$ og $(0.4)_2$. $(141.4)_8$ følger av å gruppere bitene på tre og tre, dvs. $(141.4)_8 = |010|001|101|. |011|001|100|110|$.

Normaliserte reelle tall (såkalte flyttall) uttrykkes matematisk på formen

$$\pm 1.\underbrace{bbbbbbbbbb}_{\text{mantissa}} \dots \times 2^e$$

($b = 0/1$; eksponent e),¹ men hvordan representeres de i datamaskinen?

Når du for eksempel taster inn et tall på terminalen og trykker enter, hva blir lagret internt?

Datamaskiner bruker (selvsagt) et endelig antall bits (0/1) for å lagre tall.

Som vi skal se, kan derfor regneoperasjoner gi oss resultater som både kan overraske og føre til store feil. **Prøv selv $0.1 + 0.1 + 0.1$ i python!**²

Å være oppmerksom på, og kunne kontrollere dette, er en viktig del av fagfeltet numerisk analyse.

¹ Som indikert tidligere, kan det her være uendelig mange bits.

² Python returnerer 0.30000000000000004 (med 15 0-er). (Gir vi 0.3 som input returnerer Python 0.3.)

$$(1.3)_{10} = +\underline{1}.3 \times 10^0$$

$$(0.13)_{10} = +1.3 \times 10^{-1}$$

$$(13)_{10} = +\underline{1}.3 \times 10^1$$

$$(0.1)_2 = +1.0 \times 2^{-1}$$

$$(1.0)_2 = +1.0 \times 2^0$$

$$(10)_2 = +1.0 \times 2^1$$

Binær-representasjon av reelle tall

Binær-representasjon av reelle tall har kan ha **vesentlig forskjellige egenskaper** ettersom de er rasjonale eller irrasjonale tall.

Vi har:

SETNING

For r et reelt tall gjelder det at r er en brøk hvis og bare hvis r har enten en endelig eller en (uendelig) da periodisk binær-representasjon (b-repr.).

Et irrasjonalt tall har derfor en (uendelig) ikke-periodisk b-repr..

En brøk $r = a/b$ med $\gcd(a, b) = 1$ (dvs. en mest mulig forkortet brøk)¹ har en endelig b-repr. hvis og bare hvis 2 er den eneste primtallsfaktoren til b eller $b = 1$ (for $a/b = a$ heltall).

For rasjonale tall hjelper setningen oss til å **kontrollere om vi har fullført konverteringen** eller ikke. (Har vi riktig antall bits for heltall og har vi riktig identifisering (lengde) av den gjentakende perioden ellers.) **TIPS: Kontrollér alltid binær-representasjonen din!**

¹UTAG kan vi anta at a og b er positive heltall.

1. Evaluering av polynom (bla. Horners metode)
2. Binære tall - konvertering mellom desimal tall og binære tall (begge veier)
3. Andre tallsystem
4. Reelle tall representert som flyttall (del 1)

Igjen, normaliserte reelle tall (såkalte flyttall) uttrykkes matematisk på formen $\pm 1.bbbbbbbbbbbb \dots \times 2^e = \pm m \times 2^e$

Her er $b = 0/1$; eksponenten e er heltallig og mantissen m oppfyller $1 \leq m < 2$.¹

EKSEMPEL

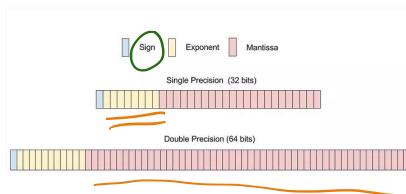
$$n = 23 = (10111)_2 = (1.0111)_2 \times 2^4.$$

Her er altså $e = 4$ og $m = \frac{23}{16} = 1 + \frac{7}{16} = (1.4375)_{10} = (1.0111)_2$.²

¹Merk at m inkluderer 1. delen.

²Her er $7/16 = 1/4 + 1/8 + 1/16$.

Merk at IEEE 754 modellene ble først tilgjengelige fra slutten av 1980-tallet!



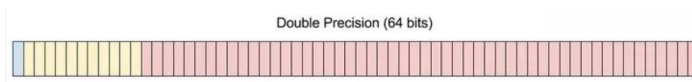
SP: S(1), E(8) og M(23) [til sammen 32 biter] / DP: S(1), E(11) og M(52) [til sammen 64 biter]

Endelig representasjon av reelle tall på datamaskinen vil ofte føre til avrundsfeil.

Hvis mange nok flyttallsoperasjoner utføres, så kan denne feiltypen akkumuleres og i verste fall fullstendig ødelegge beregningene.

Vi skal konsentrere oss mest om DP.¹

¹Vi har også andre IEEE 754 formater, som foreks. "longdouble" med S(1), E(15) og M(64).



Maskin-representasjon (dobbel presisjon)

Flyttall med dobbel presisjon lagres med 64 bits i en datamaskin.

- 1 bit s til fortegnet.
 - ▶ $s = 0$ betyr positivt tall.
 - ▶ $s = 1$ betyr negativt tall.

- 11 bit til eksponenten e .

- ▶ $-1022 \leq e \leq 1023$

- ▶ Tallet $e + 1023$ lagres.

1023 bias

- ▶ Eksponentbias $1023 = (011\ 1111\ 1111)_2$ trekkes fra for å få tilbake e

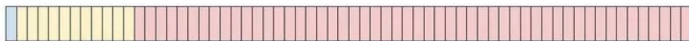
- 52 bit til mantissa $1 \leq m < 2$.

- ▶ Heltallet $M = 2^{52}(m - 1)$ lagres.

- ▶ Mantissa regnes ut med $m = 1 + M \cdot 2^{-52}$.

//
= ~

Double Precision (64 bits)

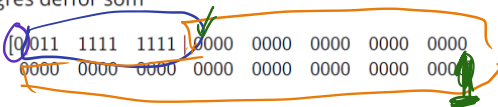


Tallet 1 i dobbel presisjon

Vi skriver $1 = +1 \times 2^0$

- Derfor $s = 0$, $e = 0$ og $m = 1$.
- Vi legger til eksponentbias $1023 = (011\ 1111\ 111)_2$ til eksponenten e .
- Heltallet $M = 2^{52}(m - 1) = 0$ lagres for mantissa.

Tallet 1 lagres derfor som



= 3FF0 0000 0000 0000

på hexadecimal-form.

Eksempel: $n = \pi$ (π er et irrasjonalt tall)¹

$\pi =$

3.14159265358979323846264338327950288419716939937510 ... =
(11.001001000011111101101010100010001000010110100011 ...)₂

Uendelig og ikke periodisk binær-representasjon.

Lagret i double precision IEEE 754 format (64 bits) ekvivalent med
400921FB54442D18 (hex)

¹Andre kjente irrasjonale tall er for eksempel $\sqrt{2}$ og e .

$$1 + 2^{-53} = 1$$

1) Merk at 2^{-53} er det største maskinrepresentative tallet mindre enn $\epsilon_M = 2^{-52}$ (machine epsilon) som oppfyller at flyttallet $1 + 2^{-53}$ er lik 1:

$$1 + \boxed{000000000000|0000000000000000\ldots 00000000|1000\ldots} = 1$$

2) Merk også at $1 + \epsilon_M = 1 + 2^{-52}$ er det minste flyttallet større enn 1:

$$1 + \boxed{000000000000|0000000000000000\ldots 00000001|0000\ldots} > 1$$

Vi skal senere gå gjennom avrundingsreglene som bla. forklarer relasjonene til 1 i 1) og 2).

35 / 51

Ofte er det ikke nok med 52 bits i mantissa for å representere tallene eksakt.

Hvordan avrunder vi til 52 bits når det er nødvendig?

Men, er avrundingsfeil noe å bry seg om på moderne datamaskiner?

Rounding errors are inevitable when finite-precision computer memory locations are used to represent real, infinite precision numbers. Although we would hope that small errors made during a long calculation have only a minor effect on the answer, this turns out to be wishful thinking in many cases. **Simple algorithms, such as Gaussian elimination or methods for solving differential equations, can magnify microscopic errors to macroscopic size.** In fact, a main theme of this book is to help the reader to recognize when a calculation is at risk of being unreliable due to magnification of the small errors made by digital computers and to know how to avoid or minimize the risk.

Det ovenfor er sakset fra SAUER, s. 8.

Merk den siste setningen ovenfor.

Om avrunding i IEEE 754 DP modellen

Feilskranker for absolutt feil og relativ feil

Gitt den matematiske normaliserte flyttallsrepresentasjonen av det positive reelle tallet

$$x = 1.\text{ bbbbbb } \dots \text{ bbbbbb } \text{ bbbbbb } \dots \times 2^e.$$

Når vi representerer x i datamaskina (som $fl(x)$) i IEEE 754 DP modellen, hvilken feil gjøres da?

Vi har da at x ligger mellom sine to nærmeste flyttall $fl_L(x)$ og $fl_U(x)$ som definert nedenfor, og illustrert for IEEE 754 DP modellen.

Nå velges $fl(x)$ som enten $fl_L(x)$ eller $fl_U(x)$ etter den gjeldende avrundingsregelen, som det nærmeste av $fl_L(x)$ og $fl_U(x)$.

Så, $fl_L \leq x \leq fl_U(x)$ der

$$fl_L(x) = 1. \boxed{b b b b b b \dots b b b b b b} \times 2^e \text{ og } /$$
$$fl_U(x) = (1. \boxed{b b b b b b \dots b b b b b b} + \underbrace{2^{-52}}_{\uparrow}) \times 2^e.$$

Dette gir da at den absolutte feilen oppfyller

$$|fl(x) - x| \leq (fl_U(x) - fl_L(x))/2 = 2^{-52}/2 \times 2^e = 2^{-53} \times 2^e.$$

Da også $|x| \geq 2^e$ (hvorfor?), vil den relative feilen oppfylle

$$|fl(x) - x|/|x| \leq 2^{-53} = \epsilon_M/2 \text{ med } \epsilon_M = 2^{-52}.$$

Dette er feilskranken (0.9) for **den relative avrundingsfeilen** på side 10 i SAUER. (SAUER gir ikke noe bevis for dette.)

Men, jeg syns dette er veldig instruktivt å ta med.

IEEE Rounding to Nearest Rule

For double precision, if the 53rd bit to the right of the binary point is 0, then round down (truncate after the 52nd bit). If the 53rd bit is 1, then round up (add 1 to the 52 bit), unless all known bits to the right of the 1 are 0's, in which case 1 is added to bit 52 if and only if bit 52 is 1.

Avrundingsreglene ovenfor er sakset fra SAUER, s. 10.

Vi skal vise at dette gjør at $\text{fl}(x)$ for gitt x er **det nærmeste flyttallet til x unntatt når x ligger like langt unna de to nærmeste flyttallene til x (unntaksregelen)**.

Vi skiller mellom følgende tilfeller etter bit 53 i mantissa:

- 1) **1000000**...
- 2) **0bbbbbb**...
- 3) **1bbbbbb**... der 3) er forskjellig fra det som er tilfellet 1

Avrundingsregler i IEEE 754 DP modellen

La igjen $x = 1.$ **bbbbbb** ... **bbbbbb**|**bbbbbb**... $\times 2^e$.

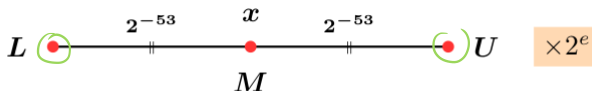
Hvordan velges $fl(x)$ mellom $fl_L(x) = L \times 2^e$ eller $fl_U(x) = U \times 2^e$ der $U = L + 2^{-52}$ ved avrundingen?

Når det gjelder avrundingen påvirker denne kun mantissa **bbbbbb** ... **bbbbbb**.

I tilfelle 1 (tilfellet med **1000000**... for bitene fra og med bit 53) i representasjonen av x , så **ligger x like langt fra de to flyttallene** da

$$M = (fl_L(x) + fl_U(x))/2 = (L + 2^{-53}) \times 2^e = (U - 2^{-53}) \times 2^e = x.$$

Unntakstilfellet



Her er det definert en **unntaksregel** for hva $fl(x)$ skal være da begge alternativene $fl_L(x)$ og $fl_U(x)$ er jo like gode tilnærminger til denne x (som vist ovenfor):

Avrund slik at det siste sifferet (bit nr. 52 i mantissa til x) alltid blir 0.

Dette gjøres for å forsøke å hindre en uønsket (biased) effekt (oppbygning) av de akkumulerte avrundingsfeilene i beregninger.

Husk mentetall når en avrunder oppover!

Dette gjøres for å forsøke å hindre en uønsket (biased) effekt (oppbygning) av de akkumulerte avrundingsfeilene i beregninger.

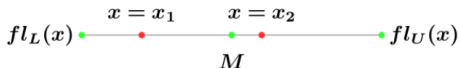
Da gjenstår det to tilfeller for x der bitene fra og med 53 for x er 2) **0bbbbbb**... eller 3) **1bbbbbb**... der 3) er forskjellig fra det som ga tilfellet 1 (unntakstilfellet) **1000000**... ovenfor.

I tilfelle 2) (**0bbbbbb**...) har vi $x = x_1 < M$, så x ligger nærmere $fl_L(x)$ enn $fl_U(x)$, og $fl(x)$ settes derfor til $fl_L(x)$. Dette stemmer overens med at det her ikke blir noen avrunding, dvs. **alle biter etter bit 52 i x ignoreres**. (Vi har det som kalles chopping.)

I tilfelle 3) (**1bbbbbb**...) har vi $x = x_2 > M$.

Så, x ligger nå nærmere $fl_U(x)$ enn $fl_L(x)$.

Altså settes $fl(x)$ da til $fl_U(x)$, så derfor **avrundes x opp**.



Et gjensyn.

La F være alle flyttall representert i IEEE 754 DP modellen.

Merk at det reelle tallet $x = 1 + 2^{-53} \notin F$.

Det ligger midt i mellom sine to nærmeste flyttallene $1 \in F$ og $1 + 2^{-52} \in F$.

Unntaksregelen for avrunding gir derfor at $fl(x) = 1$.

$1 + 2^{-53}$:

$$1 + \boxed{\text{000000000000} | \text{0000000000000000} \dots \text{00000000} | \text{1000} \dots} = 1$$

Vi skal bestemme IEEE 754 DP representasjonene av $x = \pm 0.1$ og ± 0.2 .

	n	Rasjonal del av 2n	Heltallsdel av 2n
2	0.1	0.2	0
2	0.2	0.4	0
2	0.4	0.8	0
2	0.8	0.6	1
2	0.6	0.2	1
2	0.2	0.4	0

n	n/2	Integer part module 2	+	Rest module 2
1019	1019/2	509	+	1
509	509/2	254	+	1
254	254/2	127	+	0
127	127/2	63	+	1
63	63/2	31	+	1
31	31/2	15	+	1
15	15/2	7	+	1
7	7/2	3	+	1
3	3/2	1	+	1
1	1/2	0	+	1

Først så har vi av venstre tabell at $x = 0.1 = (0.00011)_2$.

Altså er den normaliserte flyttallsrepresentasjonen av $x = 0.1$ lik $1.(\overline{1001})_2 \times 2^{-4}$.

Dette gir at eksponenten er $e = -4$ med biased exponentlik $BE = 1023 - 4 = 1019 = (01111111011)_2$ (11 biter i BE (les siste kolonne i tabellen til høyre baklengs))¹

Da representeres 0.1 i IEEE 754 DP formatet som²

001111111011 | **1001100110011001100110011001100110011001100110011010** .

¹I python gir `bin(1019,2)` dette direkte.

²Det har vært en avrunding oppover som har endret de to siste bitene.

Gjentar for ***fI(0.1)***:

001111111011		10011001100110011001100110011001100110011010
--------------	--	--

Eneste som skiller representasjonene av 0.1 og -0.1 er at for -0.1 er første bit (fortegns-biten) lik 1 mens den er 0 for 0.1.

Og, det som skiller ± 0.2 fra ± 0.1 er at BE øker med 1 (da e øker med 1).

Vi får at i 16-tallssystemet representeres 0.1 og -0.1 som henholdsvis ***3FB9999999999999A*** og ***BFB999999999999999A*** (med 12 niere).

Representasjonene av ± 0.2 er henholdsvis ***3FC9999999999999A*** og ***BFC999999999999999A*** da

$$BE_{0.2} = \boxed{001111111011} + 1 = \boxed{001111111100} \quad (1)$$

$$BE_{-0.2} = \boxed{101111111011} + 1 = \boxed{101111111100} . \quad (2)$$

16-tallssystemets spesialtegn:

$$A = 1010 / B = 1011 / C = 1100 / D = 1101 / E = 1110 / F = 1111$$

Biased Exponent BE

Biased betyr **forutantatt** på norsk og vi har her en regel ($BE = e + 1023$) som er valgt for hvordan vi skal representere den vanlige eksponenten e med BE uten at vi trenger å lagre fortegnet til e . Vi ser at DP formatet dekker et stort spenn på eksponenten.

Exponents

- ▶ 11 bits allow us to represent $2^{11} = 2048$ different integers
- ▶ We need both positive and negative exponents, and we keep two integers for special cases, so we have 2046 left to cover integers from -1022 to 1023:

binary	00000000000	00000000001	00000000010	...	11111111110	11111111111
decimal	0	1	2	...	2046	2047
exponent	special	-1022	-1021	...	1023	special

- ▶ 1023 is the **exponent bias**. We add this amount to the actual exponent, and then store that number (note: this is the same as adding 1024 and subtracting 1)
- ▶ Subtract 1023 after storing (or subtract 1024 and add 1)

Vi vil ikke gå nøyere inn på de spesielle eksponentene (0 og 2047).
(Men, dere kan lese om dette i SAUER 0.3.2, s. 12-13.) **(IKKE eksamensstoff!)**

New conversion

0 10000000010 00101100110011001100110011001100110011001100110011001101

ALGORITME

Floating point binary addition

- Make sure both numbers are normalised
- Make smaller exponent match the larger exponent
- Add mantissas together
- Normalise result if necessary

Machine addition consists of lining up the decimal points of the two numbers to be added, adding them, and then storing the result again as a floating point number. The addition itself can be done in higher precision (with more than 52 bits) since it takes place in a register dedicated just to that purpose. Following the addition, the result must be rounded back to 52 bits beyond the binary point for storage as a machine number.

[illegible]

Vi går ikke inn på algoritmene for $-$, $\times$ og $/$ av flyttall.

Addering av flyttall er **ikke en assosiativ operasjon**:¹

Prøv uttrykkene (1) $(x + y) + z$ og (2) $x + (y + z)$ for $x = 0.1$, $y = 0.3$ og $z = 0.00001$ i MATLAB eller Python

```
$ python
Python 2.7.17 (default, Mar  8 2023, 18:40:28)
[GCC 7.5.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> x=0.1
>>> y=0.3
>>> z=0.00001
>>> (x+y)+z
0.400010000000000003
>>> x+(y+z)
0.40001
```

¹Det gjelder heller ikke for subtraksjon, multiplikasjon og divisjon. Men, merk at de elementære regneoperasjonene er kommutative.

- 1) De spesielle tallene ($\pm\text{Inf}$ og **NaN**) forsikrer at flyttallsaritmetikken gir et veldefinert flyttall og ikke vil føre til at programmet pr. default stopper opp eller kræsjer. Dette er viktig særlig for kritiske sanntidssimuleringer.
- 2) Også de spesielle verdier som returneres i unntakstilfeller er designet til å gi "korrekte" (gode) svar i de fleste tilfeller.
- 3) De små tallene (subnormal numbers) forsikrer også at for endelige flyttall x og y er $x-y = 0$ ekvivalent med $x = y$, som en kunne forvente, **men som ikke var tilfelle før IEEE 754.**
- 4) **Tallene ovenfor er ikke nyttig kun for ekspertene, men forbedrer hverdagen for den jevne programmerer.**

Vi går som sagt ikke inn på hvordan disse spesielle og små tallene er representert i IEEE 754 DP modellen.